

Računarska grafika
(3OER7O02)

Koordinatni prostori i transformacije

Vežbe



Koordinatni prostori i transformacije

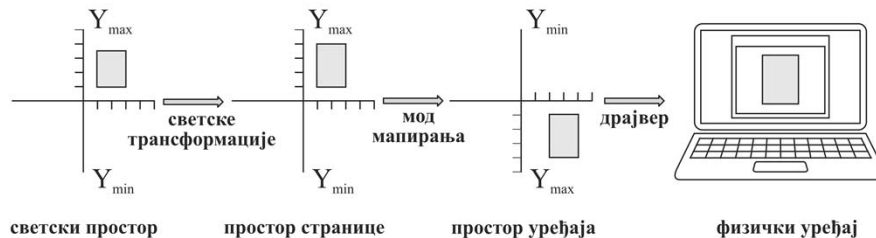
- Koordinatni prostori i transformacije koriste se za skaliranje, rotaciju, translaciju, ...
- Koordinatni prostor je ravan prostor u kome se koristi Dekartov koordinatni sistem.
- Postoje 4 koordinatna prostora:
 - **world**
 - **page**
 - **device** i
 - **physical device**
- Transformacija je algoritam po kome se menja veličina, orijentacija, položaj i oblik objekata.

Transformacija prostora

Koordinatni prostor predstavlja sredstvo da se specificira lokacija svake tačke u ravni. Koriste se:

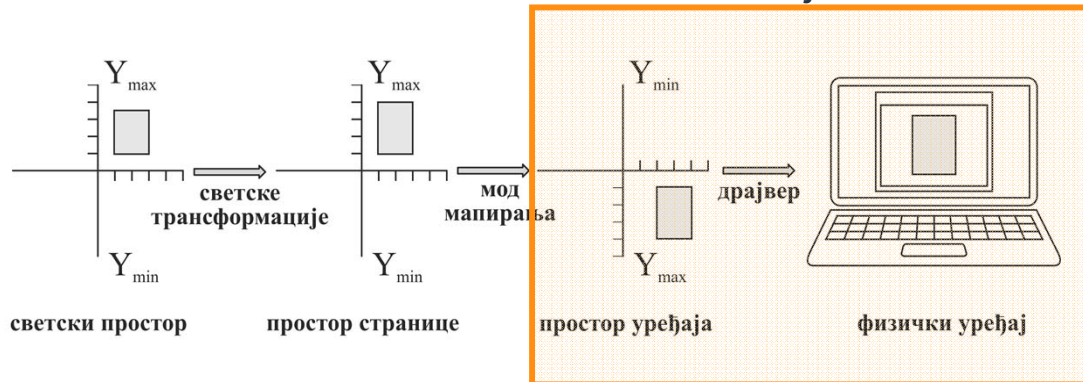
- **World** – dimenzija $2^{32} \times 2^{32}$
- **Page** – dimenzija $2^{32} \times 2^{32}$
- **Device** – dimenzija $2^{27} \times 2^{27}$
- **Physical device** – dimenzije zavise od uređaja (ekran, deo prozora, papir štampača i sl.)

Transformacija određuje način preslikavanja piksela iz jednog prostora u drugi.



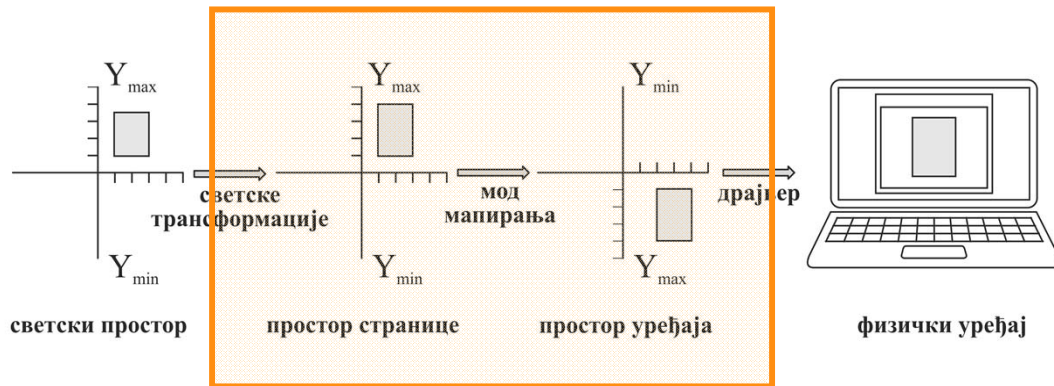
Preslikavanje *Device* u *Physical Device* prostor

- Izvršava se samo translacija
- Ovom transformacijom upravlja USER komponenta Windows-a za upravljanje prozorima
- Jedina uloga ove transformacije je da se početak *device* prostora mapira na početak (tj. odgovarajući piksel) *physical device* prostora
- Programski se ne može uticati na ovu transformaciju



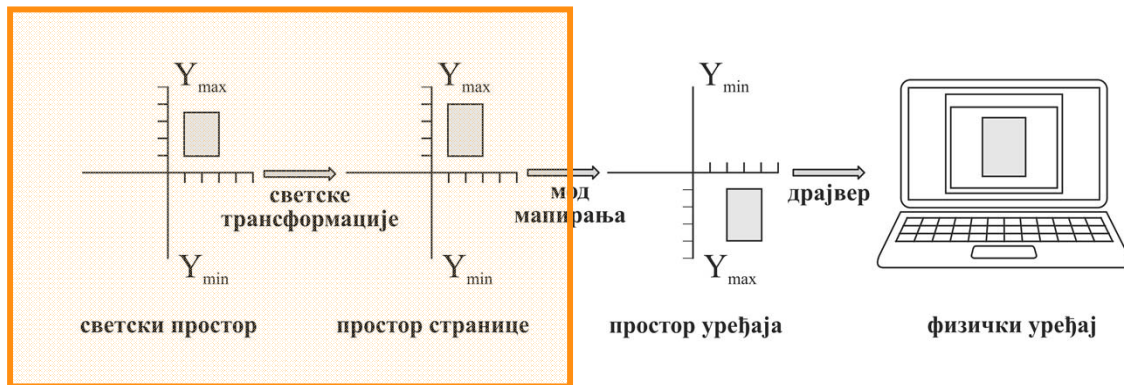
Preslikavanje *Page* prostora u *Device* prostor

- Definiše mod mapiranja za sve grafičke primitive određenog DC-ja
- Podržane su sledeće transformacije: translacija, skaliranje i refleksija (promena orijentacije osa)
- Definiše se postavljanjem odgovarajućeg moda mapiranja i koordinatnog početka



Preslikavanje *World* prostora u *Page* prostor

- Omogućava ono što se u drugim API-jima naziva preslikavanje lokalnih u svetske koordinate
- Omogućuje sledeće transformacije:
 - translaciju, skaliranje, rotaciju, smicanje, refleksiju



Modovi mapiranja

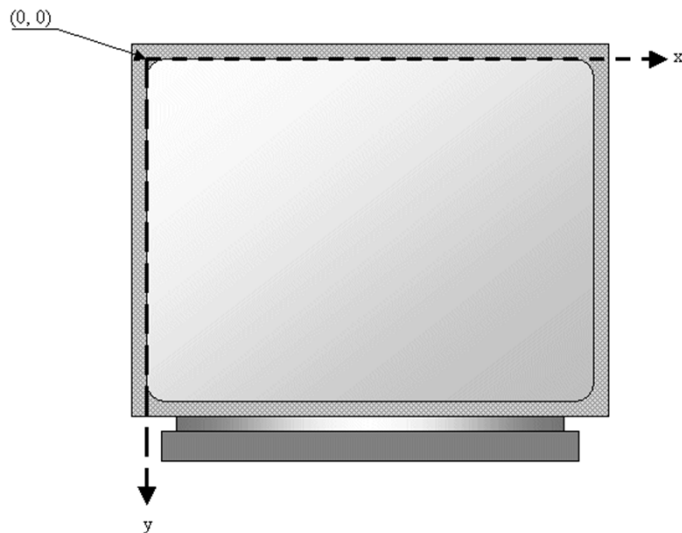
Preslikavanje prostora **stranice** u prostor **uređaja**

Mod mapiranja

- Definiše kako se koordinate mapiraju iz logičkih u fizičke.
- U **fizičkom** koordinatnom sistemu jedinice mere su uvek pikseli, y-osa raste naniže, a koordinatni početak je u gornjem levom uglu.
- Sve koordinate koje se prosleđuju funkcijama za crtanje zadaju se u **logičkom** koordinatnom sistemu.
- Inicijalno se fizički i logički koordinatni sistemi poklapaju.

Mod mapiranja – MM_TEXT

Podrazumevani mod mapiranja, u kome jednoj logičkoj jedinici odgovara jedan piksel izlaznog uređaja i koordinatne ose fizičkog i logičkog sistema se poklapaju.



Mod mapiranja – 2

Grupa modova usklađenih sa jedinicama dužine koje se koriste u svakodnevnom životu:

- **MM_TEXT** - 1 logika jedinica mapira se u 1 piksel. X raste na desno, Y naniže.
- **MM_HIENGLISH** – 1 logika jedinica mapira se u 0.001 inča. X raste na desno, Y naviše.
- **MM_HIMETRIC** - 1 logika jedinica mapira se u 0.01 mm. X raste na desno, Y naviše.
- **MM_LOENGLISH** - 1 logika jedinica mapira se u 0.01 inča. X raste na desno, Y naviše.
- **MM_LOMETRIC** - 1 logika jedinica mapira se u 0.1 mm. X raste na desno, Y naviše.
- **MM_TWIPS** - 1 logika jedinica mapira se u 1 **twip** (1/20 tipografske tačke ili 1/1440 inča). X raste na desno, Y naviše.

Mod mapiranja – 3

Proizvoljne veličine logičkih jedinica

- **MM_ISOTROPIC** - logičke jedinice mapiraju se u proizvoljne fizičke, ali sa ograničenjem da je jednako skaliranje po obe ose. (Kada se postavi *window extent*, *viewport* se automatski podešava da zadrži izotropnost - ne poštuju se u potpunosti postavke *viewport* i *window extent-a*).
- **MM_ANISOTROPIC** – logičke jedinice mapiraju se u proizvoljne fizičke.

Mod mapiranja – postavljanje

Funkcije za očitavanje trenutno selektovanog moda i postavljanje novog moda mapiranja:

```
int CDC::GetMapMode( );
```

```
int CDC::SetMapMode(int fnMapMode);
```

Mod mapiranja – funkcije preslikavanja

Postavljanje mapiranja fizičkih u logičke jedinice. Koriste se samo kod **MM_ISOTROPIC** i **MM_ANISOTROPIC** modova, uvek u paru, i definišu koliko se logičkih jedinica preslikava u jednu fizičku i obrnuto.

```
■ BOOL CDC::SetWindowExt(  
    int nXExtentW, // nova širina prozora  
    int nYExtentW, // nova visina prozora);  
  
■ BOOL CDC:: SetViewportExt(  
    int nXExtentV, // nova širina viewport-a  
    int nYExtentV, // nova visina viewport-a );
```

Značenje: ***nXExtentV*** fizičkih jedinica (piksela) odgovara ***nXExtentW*** logičkih jedinica po X-osi, a ako se znaci razlikuju X-osa raste ulevo. Isto važi i za Y-osi (ako se razlikuju znaci, Y-osa raste naviše).

Mod mapiranja – funkcije postavljanja koordinatnog početka

Koje će logičke koordinate imati tačka sa fizičkim koordinatama (0,0), zadaje se pomoću funkcije:

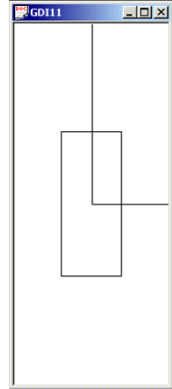
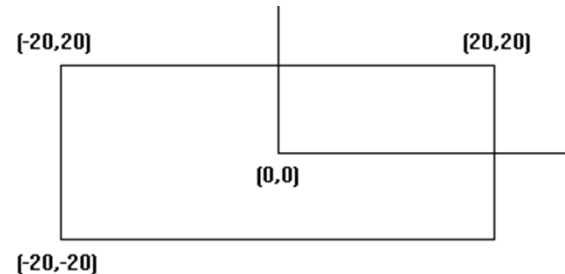
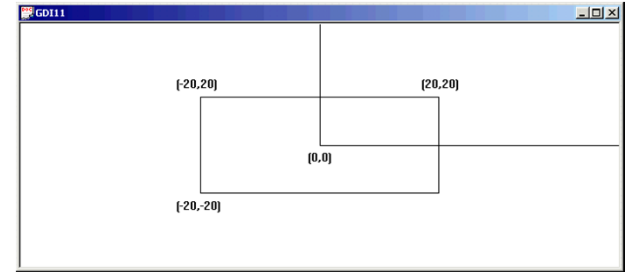
```
■ BOOL CDC::SetWindowOrg(  
    int X, // nova logička x-koordinata početka prozora  
    int Y // nova logička y-koordinata početka prozora );
```

Koje će fizičke koordinate imati tačka sa logičkim koordinatama (0,0), tj. položaj u odnosu na gornji levi ugao, zadaje se pomoću funkcije:

```
■ BOOL CDC::SetViewportOrg(  
    int X, // nova fizička x-koordinata početka viewport-a  
    int Y // nova fizička y-koordinata početka viewport-a);
```

Primer

```
CRect rect;  
GetClientRect(&rect);  
pDC->SetMapMode(MM_ANISOTROPIC);  
pDC->SetWindowExt(100,-100);  
pDC->SetViewportExt(rect.right,rect.bottom);  
pDC->SetWindowOrg(-50,50);  
pDC->Rectangle(-20,20,20,-20);
```



Geometrijske transformacije

Preslikavanje **svetskog** u prostor **stranice**

Osnovne transformacije



Иницијална фигура



Транслација



Смицање



Скалирање

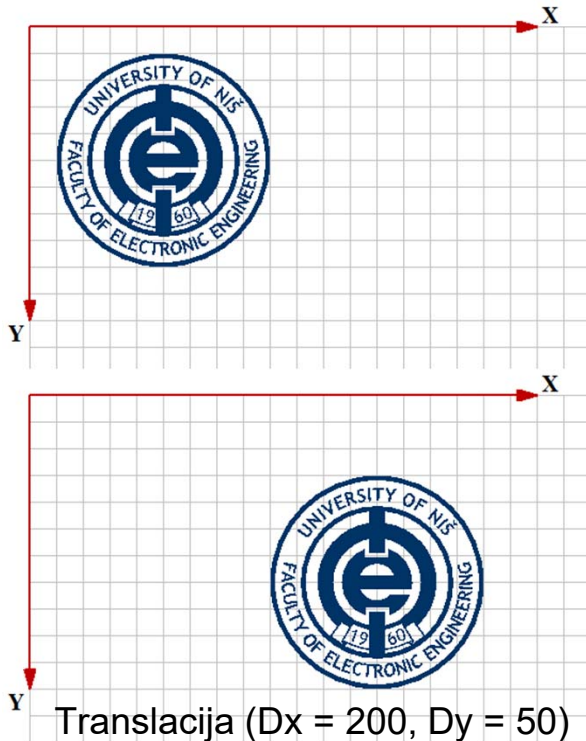


Ротација



Рефлексија

Translacija

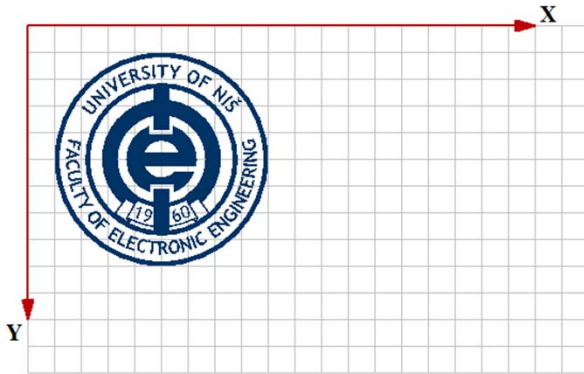


$$x' = x + Dx$$

$$y' = y + Dy$$

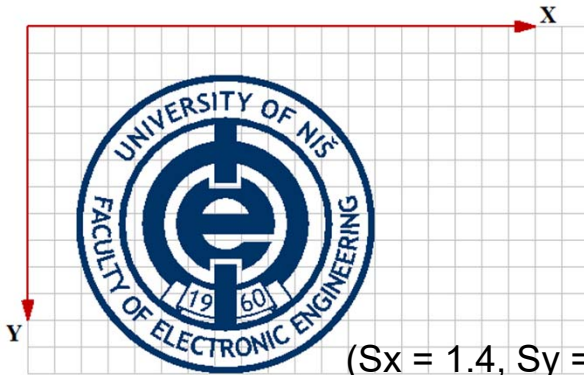
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ Dx & Dy & 1 \end{bmatrix}$$

Skaliranje



$$x' = S_x \cdot x$$

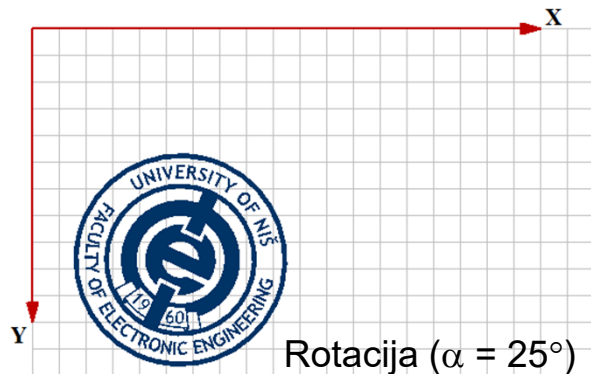
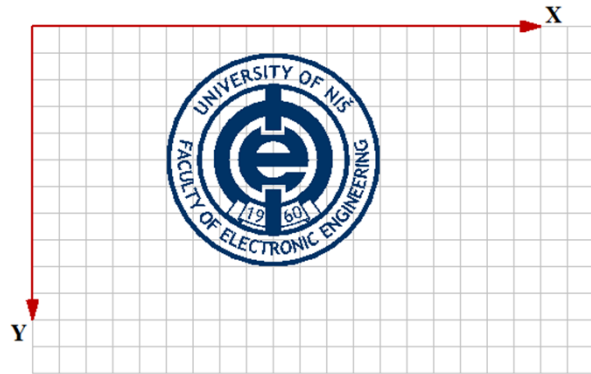
$$y' = S_y \cdot y$$



($S_x = 1.4$, $S_y = 1.4$)

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Rotacija

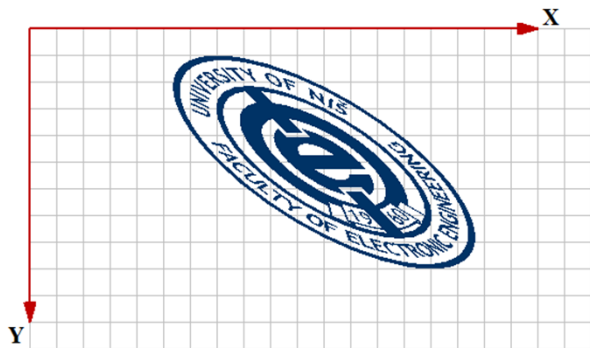
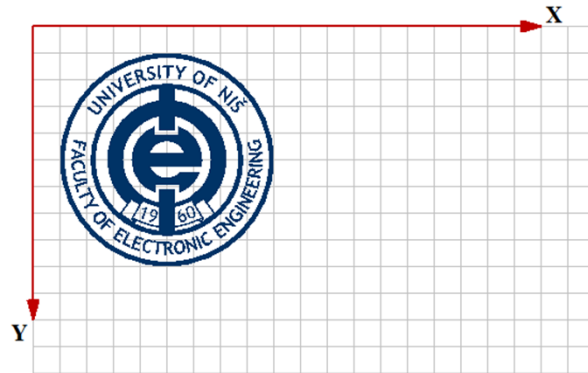


$$x' = \cos(\alpha) \cdot x - \sin(\alpha) \cdot y$$

$$y' = \sin(\alpha) \cdot x + \cos(\alpha) \cdot y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} \cos(\alpha) & \sin(\alpha) & 0 \\ -\sin(\alpha) & \cos(\alpha) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Smicanje



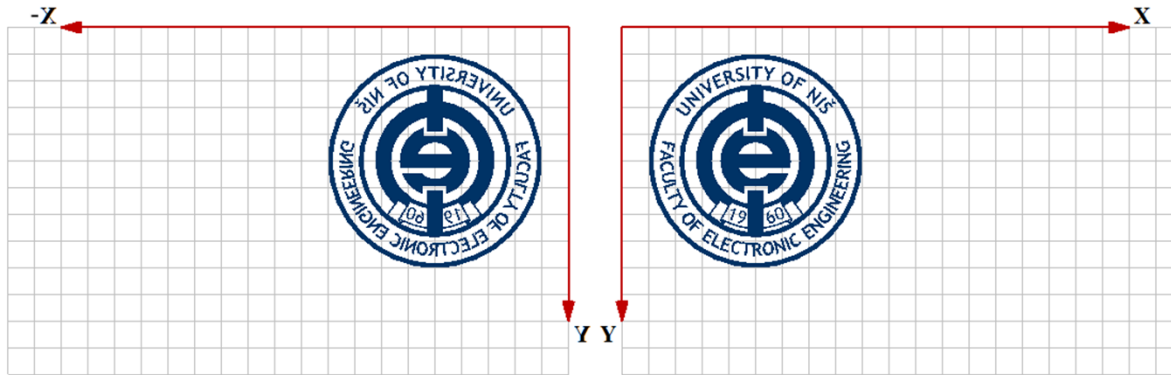
Smicanje duž X ose ($H_x = 0.0$, $H_y = 1.0$).

$$x' = x + H_x \cdot y$$

$$y' = y + H_y \cdot x$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & H_y & 0 \\ H_x & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Refleksija



Refleksija u odnosu na Y osu ($x' = -x$)

$$x' = \pm x$$

$$y' = \pm y$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} \pm 1 & 0 & 0 \\ 0 & \pm 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Struktura matrice transformacije

- XFORM struktura
- Definiše transformaciju iz *world* prostora u *page* prostor

$$x' = eM11 \cdot x + eM21 \cdot y + eDx$$

$$y' = eM12 \cdot x + eM22 \cdot y + eDy$$

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} eM11 & eM12 & 0 \\ eM21 & eM22 & 0 \\ eDx & eDy & 1 \end{bmatrix}$$

```
typedef struct _XFORM {  
    FLOAT eM11;  
    FLOAT eM12;  
    FLOAT eM21;  
    FLOAT eM22;  
    FLOAT eDx;  
    FLOAT eDy;  
} XFORM;
```

XFORM atributi

Transformacija	eM11	eM12	eM21	eM22	eDx	eDy
Translacija	1	0	0	1	Dx	Dy
Rotacija	$\cos(\alpha)$	$\sin(\alpha)$	$-\sin(\alpha)$	$\cos(\alpha)$	0	0
Skaliranje	Sx	0	0	Sy	0	0
Smicanje	1	Hx	Hy	1	0	0
Refleksija	± 1	0	0	± 1	0	0

Metoda SetWorldTransform

Postavljanje transformacije

`BOOL CDC::SetWorldTransform( const XFORM *lpXform )`

$$\begin{aligned} x' &= eM11 \cdot x + eM21 \cdot y + eDx \\ y' &= eM12 \cdot x + eM22 \cdot y + eDy \end{aligned} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} \cdot \begin{bmatrix} eM11 & eM12 & 0 \\ eM21 & eM22 & 0 \\ eDx & eDy & 1 \end{bmatrix}$$

Pre poziva ove funkcije treba postaviti grafički mod na **GM_ADVANCED**

`int CDC:: SetGraphicsMode(int iMode)`

Primer postavljanja transformacije

```
int prevMode = pDC->SetGraphicsMode(GM_ADVANCED);  
DWORD dw = GetLastError();
```

```
XFORM Xform, XformOld;  
BOOL b = pDC->GetWorldTransform(&XformOld);
```

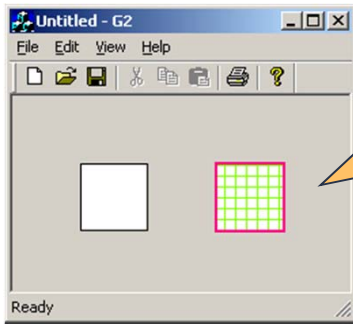
```
Xform.eM11 = (FLOAT)1.0;  
Xform.eM12 = (FLOAT)0.0;  
Xform.eM21 = (FLOAT)0.0;  
Xform.eM22 = (FLOAT)1.0;  
Xform.eDx = (FLOAT)50.0;  
Xform.eDy = (FLOAT)0.0;
```

```
b = pDC->SetWorldTransform(&Xform);
```

```
// iscrtavanje
```

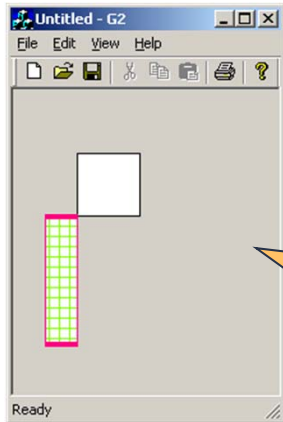
```
b = pDC->SetWorldTransform(&XformOld);  
pDC->SetGraphicsMode(prevMode);
```

Primer translacije i skaliranja



Translacija

Xform.eM11 = 1.0;
Xform.eM12 = 0.0;
Xform.eM21 = 0.0;
Xform.eM22 = 1.0;
Xform.eDx = **100.0**;
Xform.eDy = 0.0;

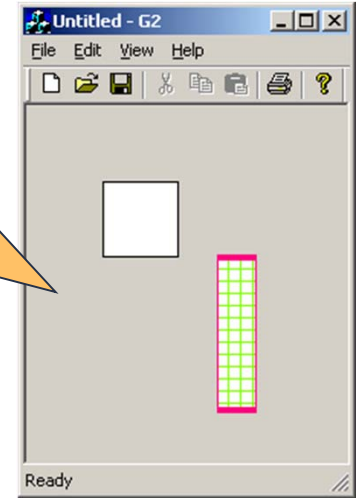


Skaliranje

Xform.eM11 = **0.5**;
Xform.eM12 = 0.0;
Xform.eM21 = 0.0;
Xform.eM22 = **2.0**;
Xform.eDx = 0.0;
Xform.eDy = 0.0;

Skaliranje i Translacija

Xform.eM11 = **0.5**;
Xform.eM12 = 0.0;
Xform.eM21 = 0.0;
Xform.eM22 = **2.0**;
Xform.eDx = **100.0**;
Xform.eDy = 0.0;



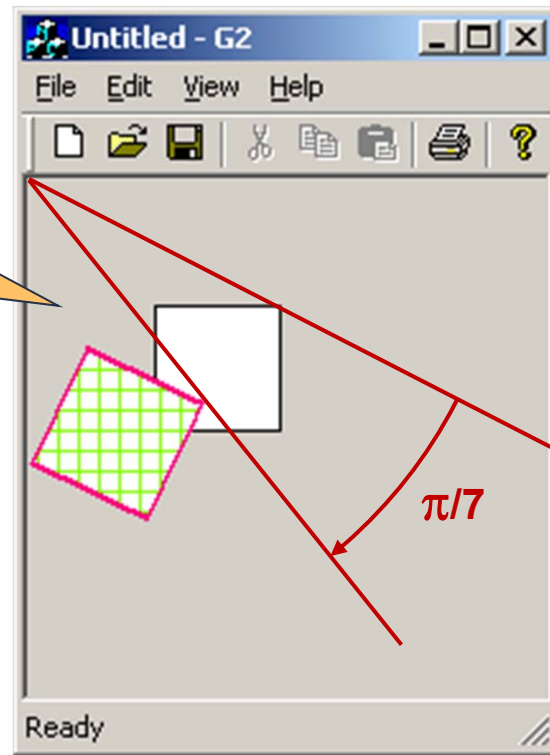
pDC->Rectangle(50,50,100,100);

Primer rotacije

Rotacija

```
Xform.eM11 = cos(pi/7.0);  
Xform.eM12 = sin(pi/7.0);  
Xform.eM21 = -sin(pi/7.0);  
Xform.eM22 = cos(pi/7.0);  
Xform.eDx = 0.0;  
Xform.eDy = 0.0;
```

```
pDC->Rectangle(50,50,100,100);
```



Kombinovanje transformacije

- Metod za izmenu trenutne transformacije DC-a

```
BOOL CDC:: ModifyWorldTransform(CONST XFORM *lpXform, DWORD iMode);
```

- lpXform* – struktura sa transformacionom matricom kojom treba izmeniti trenutnu transformaciju DC-a
- iMode* – način na koji treba izmeniti trenutnu transformaciju DC-a
 - MWT_IDENTITY** – resetuje transformaciju (učitava se jedinična matrica i ignoriše se prosleđena transformaciona matrica)
 - MWT_LEFTMULTIPLY** – množi trenutnu transformacionu matricu sa prosleđenom sa leve strane (prosleđena matrica je levi operand u množenju)
 - MWT_RIGHTMULTIPLY** – množi trenutnu transformacionu matricu sa prosleđenom sa desne strane (prosleđena matrica je desni operand u množenju)

Primer kombinovanja transformacija

```
Xform.eM11 = 1.0;  
Xform.eM12 = 0.0;  
Xform.eM21 = 0.0;  
Xform.eM22 = 1.0;  
Xform.eDx = -75.0;  
Xform.eDy = -75.0;
```

```
pDC->SetWorldTransform(&Xform);
```

```
double alpha = 45 * 3.14159 / 180;
```

```
Xform.eM11 = cos (alpha);
```

```
Xform.eM12 = sin (alpha);
```

```
Xform.eM21 = -sin (alpha);
```

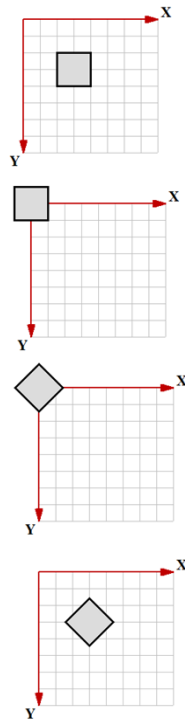
```
Xform.eM22 = cos (alpha);
```

```
Xform.eDx = 75.0;
```

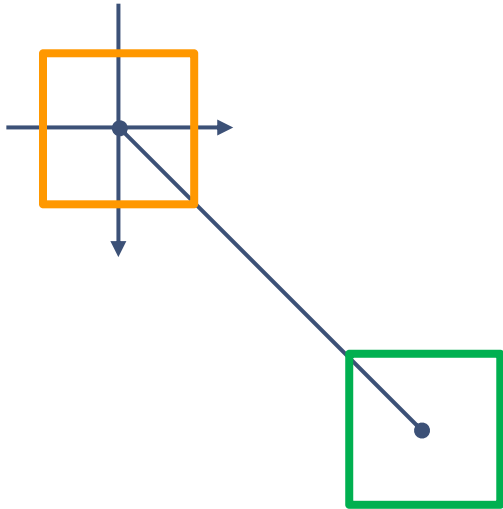
```
Xform.eDy = 75.0;
```

```
pDC->ModifyWorldTransform(&Xform,  
MWT_RIGHTMULTIPLY);
```

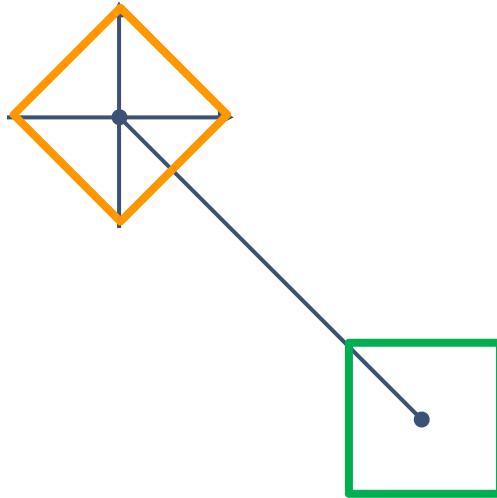
```
pDC->Rectangle(50,50,100,100);
```



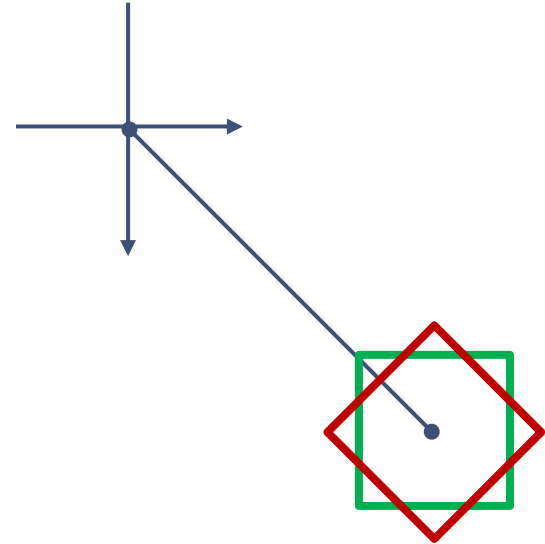
Primer kombinovanja transformacija



Translate(-75, -75)



Rotate($\pi/4$)



Translate(75, 75)

Kombinovanje transformacije

- U GDI matrica transformacija (M) množi vektor koordinata (v) sa desne strane

$$v' = v \cdot M$$

- Primena nove transformacije (M') množenjem sa leve strane (**MWT_LEFTMULTIPLY**) prethodi ranije definisanim transformacijama

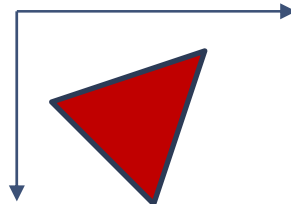
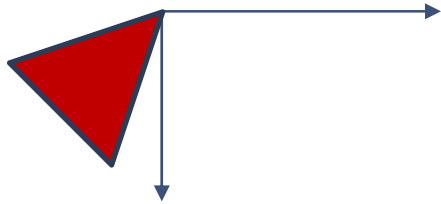
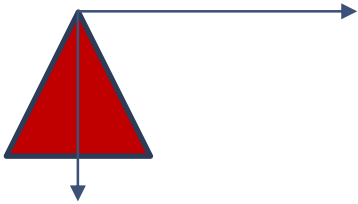
$$v' = v \cdot M' \cdot M = (v \cdot M') \cdot M$$

- Ako transformacije treba da se izvršavaju redosledom navedenim u kodu koristi se desno množenje (**MWT_RIGHTMULTIPLY**)

$$v' = v \cdot M \cdot M' = (v \cdot M) \cdot M'$$

Primer nadovezivanja transformacija

I pristup posmatranja: globalni koordinatni sistem



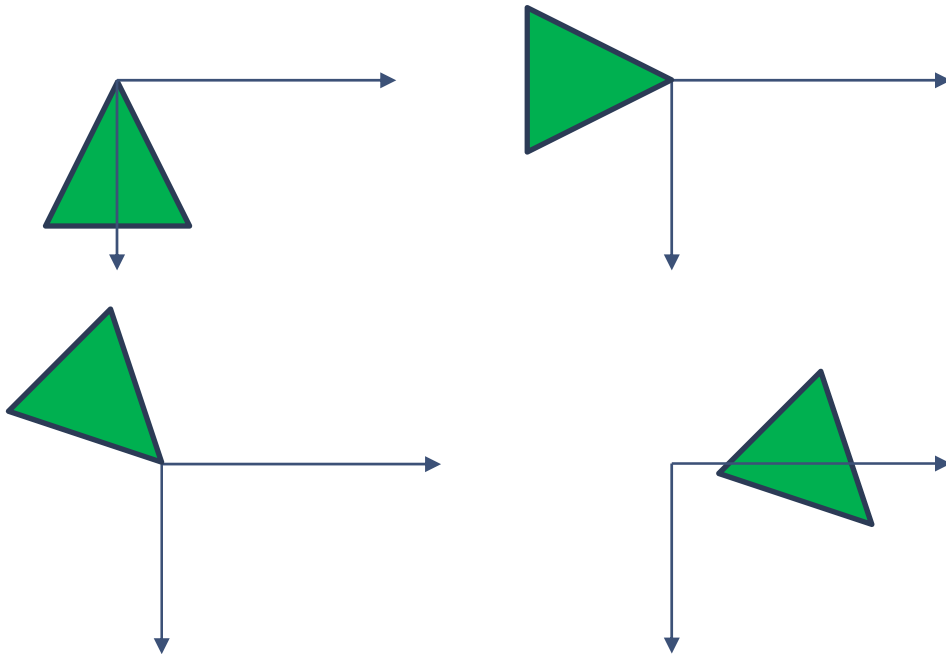
Translate(50, 10, left)

Rotate($\pi/4$, left)

// Rotate($\pi/2$, left)

Primer nadovezivanja transformacija

I pristup posmatranja: globalni koordinatni sistem



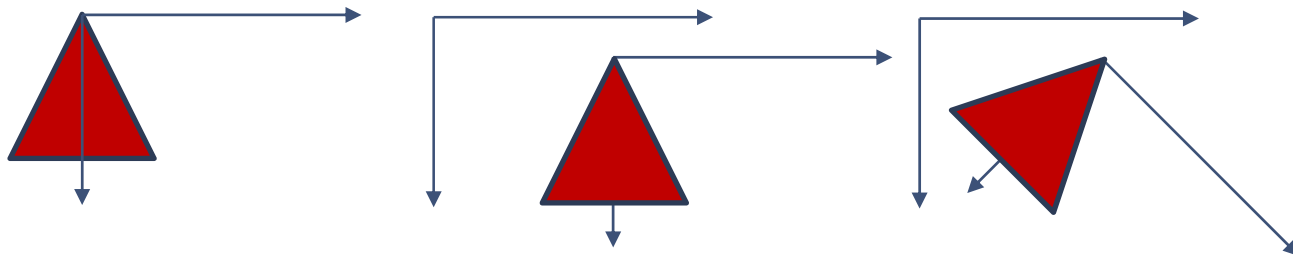
Translate(50, 10, left)

Rotate($\pi/4$, left)

Rotate($\pi/2$, left)

Primer nadovezivanja transformacija

II pristup posmatranja: lokalni koordinatni sistem



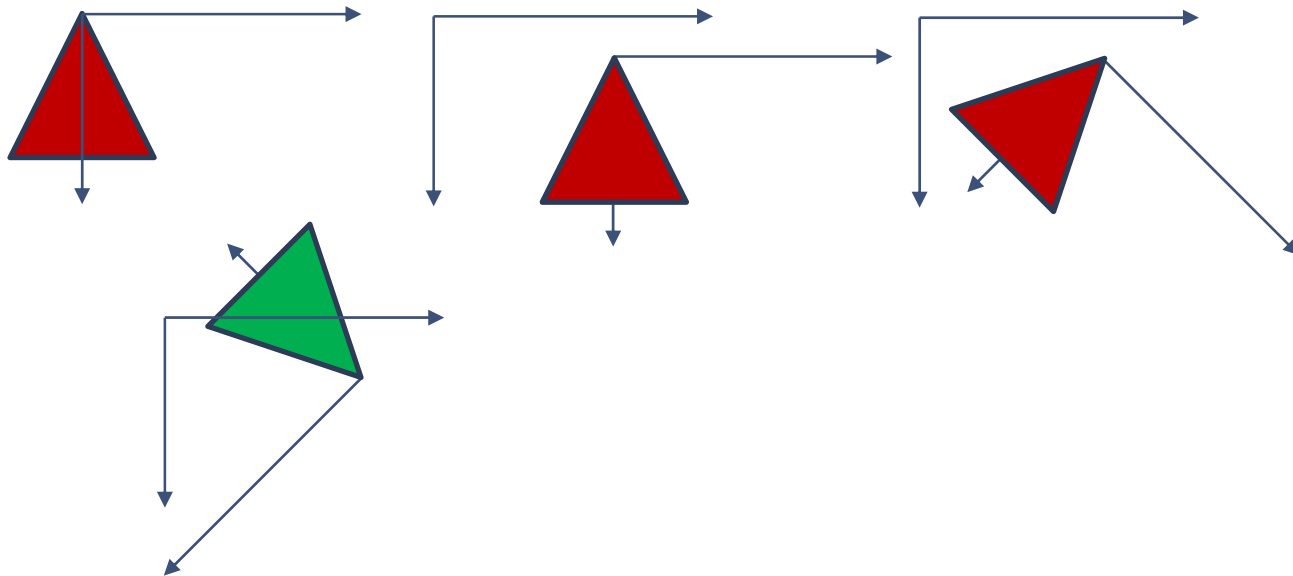
Translate(50, 10, left)

Rotate(pi/4, left)

// Rotate(pi/2, left)

Primer nadovezivanja transformacija

II pristup posmatranja: lokalni koordinatni sistem



Kombinovanje transformacija 2

- Metod za kombinovanje matrica transformacije u rezultujuću matricu

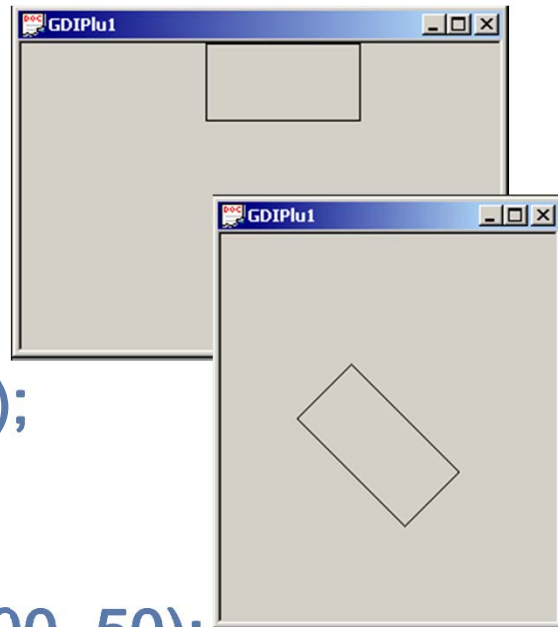
```
BOOL CombineTransform( LPXFORM lpxformResult,  
                        CONST XFORM *lpxform1,  
                        CONST XFORM *lpxform2 )
```

- lpxformResult* – pokazivač na XFORM strukturu koja prihvata kombinovanu transformaciju (rezultujuća matrica)
- lpxform1* – pokazivač na XFORM strukturu prve transformacije (leva matrica)
- lpxform2* – pokazivač na XFORM strukturu druge transformacije (desna matrica)

GDI+

Svetska transformacija *Graphics* objekta

```
Matrix transformMatrix;  
transformMatrix.Rotate(45.0f);  
graphics.SetTransform(&transformMatrix);  
Pen pen(Color(255, 0, 0, 0));  
graphics.DrawRectangle(&pen, 120, 0, 100, 50);
```



Kompozitna transformacija

```
Matrix matrix;  
matrix.Reset(); // LoadIdentity  
matrix.Translate(-X, -Y, MatrixOrderAppend);  
// MatrixOrderAppend, MatrixOrderPrepend  
matrix.Rotate(alpha, MatrixOrderAppend);  
matrix.Translate(X, Y, MatrixOrderAppend);  
graphics.SetTransform(&matrix);
```


Pitanja

